13a - CVS en R avec jd3x13 : Partie 1

Anna Smyk & Tanguy Barthélémy (Insee)

Insee
Mesurer pour comprendre

JD+
Time Series Software for Official Statistics

Sommaire I

1. Introduction

2. X13 : Lancement rapide avec les spécifications par défaut

   Récupération des résultats et visualisation des données

Section 1

Introduction

Désaisonnalisation : étapes usuelles

- tests de saisonnalité
- prétraitement
- création des variables personnalisées pour le prétraitement (par exemple, régresseurs de calendrier)
- décomposition
- récupération des séries estimées (sa, s, t, i...)
- récupération des diagnostics
- personnalisation les paramètres..au vu des diagnostics
- rafraîchissement des données (production infra-annuelle)
- ...
- répétition du processus...

Cette présentation illustrera tous ces points dans X13-Arima via RJDemetra et rjd3x13 ou rjd3tramoseats (et rjd3toolkit) .

Contexte d'utilisation

Produire des séries désaisonnalisées avec R

(avec des paramètres personnalisés en fonction des besoins et des diagnostics précédents)

- ne pas être au courant de l'existence de l'interface graphique de JD+.
- pas de structure de workspace
- objets séries temporelles dans R
- utiliser exclusivement les algorithmes de JD+ et aucun autre package R de SA (Seasonal, TBATS...)

Tous les exemples sont liés à UNE série. Pour un ensemble de données complet, vous pouvez bien sûr utiliser des boucles ou des fonctions du type `lapply()`.

Section 2

X13 : Lancement rapide avec les spécifications par défaut

Désaisonnalisation simple I

```
# X13
sa_x13_v3 <- rjd3x13::x13(y_raw, spec = "RSA5")
```

Pre-ajustement seul

```
# X13
sa_regarima_v3 <- rjd3x13::regarima(y_raw, spec = "RG5c")

# "fast." versions ...(just results, cf output structure)
```

Décomposition seule

```
# X11 is a specific function
x11_v3 <- rjd3x13::x11(y_raw) # specific function
```

Subsection 1

Récupération des résultats et visualisation des données

Récupération des résultats et visualisation des données

“Model_sa” structure

Results vs specification…and then by domain

```
# Model_sa = sa_x13_v3
sa_x13_v3 <- rjd3x13::x13(y_raw, spec = "RSA5")
sa_x13_v3$result
sa_x13_v3$estimation_spec
sa_x13_v3$result_spec
sa_x13_v3$user_defined
```

Récupérer les séries en sortie

```
# final seasonally adjusted series
sa_x13_v3$result$final$d11final
```

Séries issues du préajustement

```
# "X11 names" : pre-adjustment effects as stored in the A table
# add doc on names
sa_x13_v3$result$preadjust$a6
```

Séries issues de la décomposition

```
# Version 3
sa_x13_v3$result$decomposition$d5 # tables from D1 to D13
```

Récupération des diagnostics

Il suffit de récupérer les objets nécessaires dans la partie correspondante du modèle (liste de listes) ou de faire un `summary` du modèle.

```
print(sa_x13_v2)
sa_x13_v3$result$diagnostics$td.ftest.i
```

Ce qui manque (séries ou diagnostics) peut être récupéré en ajoutant un user-defined output dans les options

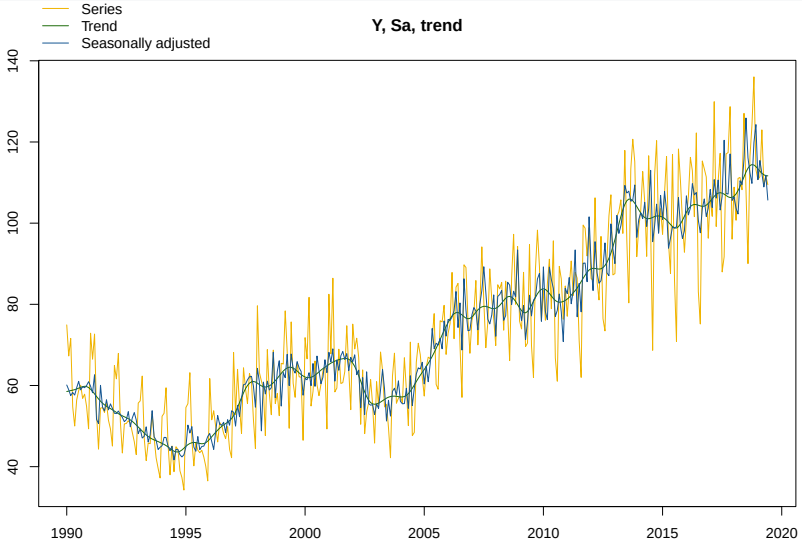
Plot de la decomposition I

```
# version 3
### ggdemetra3 for enhanced plots

# remotes::install_github("AQLT/ggdemetra3", INSTALL_opts = "--no-multiarch")
library("ggdemetra3")

# plot final decomposition
plot(sa_x13_v3)
```

Plot de la decomposition II



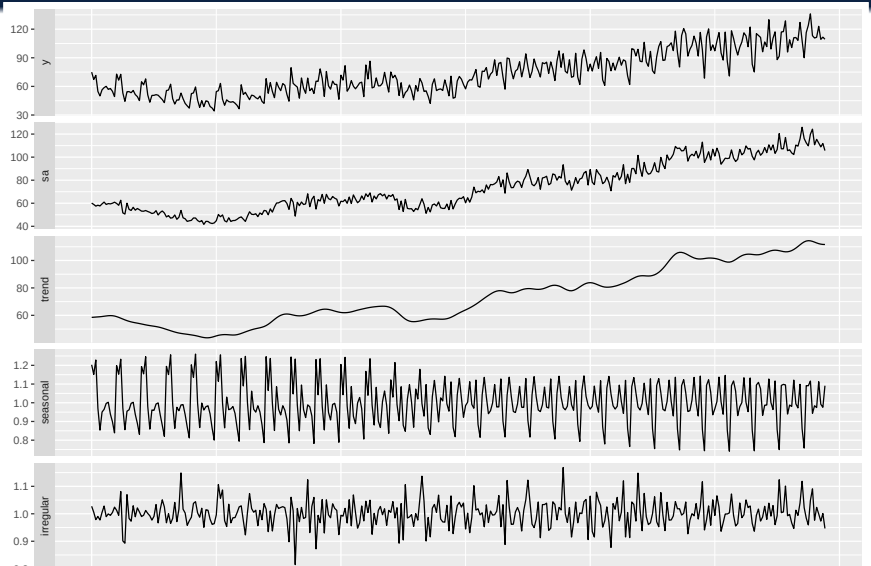
Plot de la decomposition I

New autoplot formats

```
# version 3


# in autopolot
library("ggplot2")
autoplot(sa_x13_v3)
```

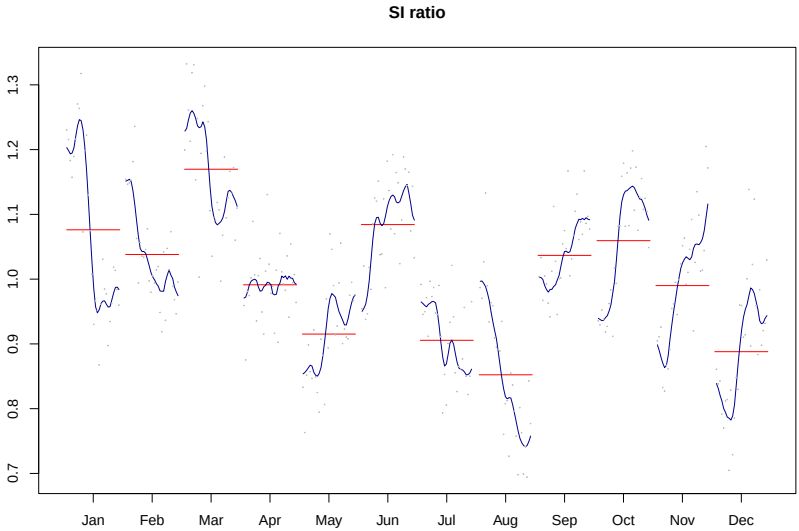
Plot de la decomposition II



Plots des SI ratios I

```
# version 3
#Plot SI ratios
siratioplot(sa_x13_v3)
```

Plots des SI ratios II

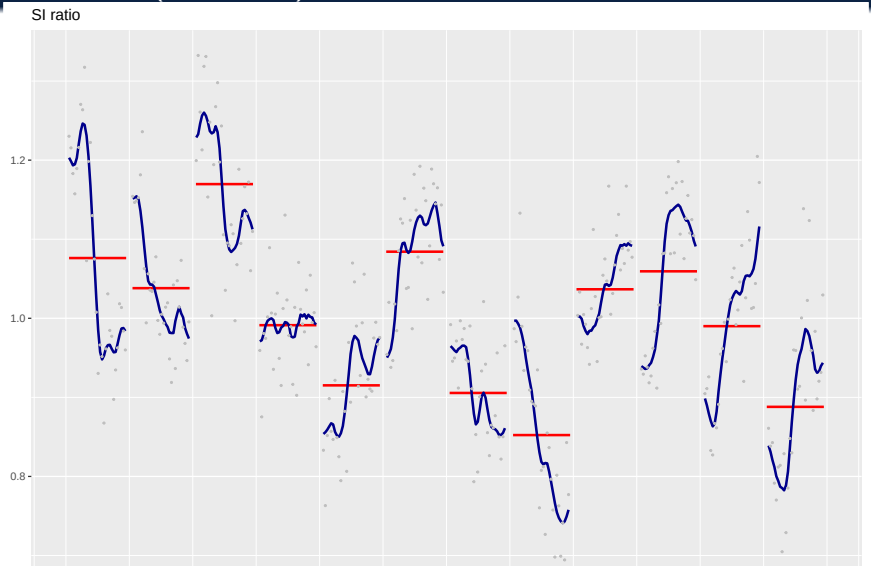


Plots des SI ratios (autoplot) I

```
# version 3
# avec le format autoplot

ggsiratioplot(sa_x13_v3)
```

Plots des SI ratios (autoplot) II



Personnalisation des spécifications : étapes générales

Pour personnaliser une spécification, vous devez

- commencer par une spécification valide, généralement l'une des spécifications par défaut (ce qui équivaut à cloner une spécification dans la GUI)
- créer une nouvelle spécification
- appliquer la nouvelle spécification à votre série de données brutes.

Personnaliser les spécifications

Utiliser des fonctions `set_` directes et spécifiques - pour l'étape de pré-traitement (fonctions définies dans `rjd3toolkit`) :

`set_arima()`, `set_automodel()`, `set_basic()`, `set_easter()`, `set_estimate()`, `set_outlier()`, `set_tradingdays()`, `set_transform()`, `add_outlier()` et `remove_outlier()`, `add_ramp()` et `remove_ramp()`, `add_usrdefvar()`.

- pour l'étape de décomposition en X13 (fonction définie dans `rjd3x13`) : `set_x11()`
- pour l'étape de Benchmarking (fonction définie dans `rjd3toolkit`) : `set_benchmarking()`

Personnalisation des spécifications : exemple

```r
# start with default spec
spec_1 <- spec_x13("RSA3")
# or start with existing spec (no extraction function needed)
# spec_1 <- sa_x13_v3_UD$estimation_spec

# set a new spec
## add outliers
spec_2 <- rjd3toolkit::add_outlier(spec_1,
    type = "AO", c("2015-01-01", "2010-01-01")
)
## set trading days
spec_3 <- rjd3toolkit::set_tradingdays(spec_2,
    option = "workingdays"
)
# set X11 options
spec_4 <- set_x11(spec_3, henderson.filter = 13)
# apply with `fast.x13` (results only)
fast_x13(y_raw, spec_4)
```

Outils pour les séries temporelles

Nouvelles fonctionnalités dans la version 3

L'esprit de la version 3 est d'offrir plus d'outils indépendants :

- tests (saisonnalité, résidus (normalité, independance), effets de calendrier résiduels) dans rjd3toolkit.
- fonctions d'autocorrélation partielle et inverse
- estimation et décomposition du modèle arima (`rjd3toolkit::ucrima_estimate()`)
- agrégation à une fréquence plus basse (`rjd3toolkit::aggregate()`)

Plus de flexibilité pour l'utilisateur car elles peuvent être appliquées à tout moment et pas seulement dans le cadre d'un traitement CVS.

L'estimation du modèle Arima est notoirement plus rapide que d'autres fonctions disponibles en R.

Tests de saisonnalité

Dans rjd3toolkit :

- Canova-Hansen (`rjd3toolkit::seasonality.canovahansen()`) NEW
- X-12 combined test (`rjd3toolkit::seasonality.combined()`)
- F-test on seasonal dummies (`rjd3toolkit::seasonality.f()`)
- Friedman Seasonality Test (`rjd3toolkit::seasonality.friedman()`)
- Kruskall-Wallis Seasonality Test (`rjd3toolkit::seasonality.kruskalwallis()`)
- Periodogram Seasonality Test (`rjd3toolkit::seasonality.periodogram()`)
- QS Seasonality Test (`rjd3toolkit::seasonality.qs()`)